Workshop Lab Guide

Troubleshooting Canvas Apps Runtime Issues

Power Platform — End-to-End Hands-On Lab

April 2026

🗔 Scenario & Background

You are part of an internal IT development team responsible for maintaining an **Incident Reporting Application** used by employees to submit technical support requests.

> 💡 **Why this matters:** In real-world Power Platform deployments — especially in government and enterprise environments — Canvas Apps often break silently. Understanding why they fail and how to fix them is a critical skill for any Power Platform developer or administrator.

Recently, users report:

- App loads slowly
- Records are not saving
- Gallery data is missing
- Form shows "No item to display"
- App freezes during submission

Goal	Description
✅ Identify	Find the root cause of each issue
✅ Diagnose	Use tools like Monitor and App Checker
✅ Troubleshoot	Apply targeted fixes
✅ Optimize	Improve performance and reliability
✅ Stabilize	Ensure the app behaves predictably

🔬 Lab 1 — Patch Runtime Failure

🎯 Objective

Fix Patch failures caused by mismatched data types — one of the most common silent failures in Canvas Apps.

> 💡 **Why this lab exists:** The Patch() function is Power Apps' primary way to write data. When data types don't match the expected column type, the write operation silently fails — no error, no warning. Students need to understand why types matter and how to enforce them.

☑ Exercise 1.1 — Create Sample Data

> 💡 **What you are doing:** You are using ClearCollect() to build an in-memory collection that simulates a data source. This lets you practice without needing a live SharePoint list or Dataverse table.
>
> 💡 **Why ClearCollect?** ClearCollect() first clears the collection, then populates it — preventing duplicate records when the app restarts. It runs at app startup via OnStart, making data available to every screen immediately.

1. Select the App object in Tree View
2. Select the OnStart property
3. Paste the following formula:

```
ClearCollect(
    IncidentCollection,
    {ID: 1, Title: "VPN Failure",   Priority: 1},
    {ID: 2, Title: "Email Issue",   Priority: 2}
)
```

4. 4. Click Run OnStart to execute immediately without reloading the app.

> ⚠️ **Notice:** Priority is defined as a NUMBER (1, 2). This is intentional — you will use this in Exercise 1.4 to understand the type mismatch problem.

☑ Exercise 1.2 — Add Form Controls

> 💡 **What you are doing:** You are building a simple data entry form with two input fields and a submit button. This simulates what end-users interact with in a real incident reporting app.

Insert the following controls:

Control	Type	Name
Input field	TextInput	txtIncidentTitle
Input field	TextInput	txtPriority
Button	Button	btnSubmit

💡 **Naming convention matters:** Using descriptive prefixes (txt, btn) is a best practice. It makes formulas easier to read and debug, especially as apps grow in complexity.

Set the Button OnSelect property to:

```
Patch(
    IncidentCollection,
    Defaults(IncidentCollection),
    {
        Title:    txtIncidentTitle.Text,
        Priority: txtPriority.Text
    }
)
```

💡 **What is Defaults()?** Defaults() creates a new blank record template based on the collection or data source schema. This tells Patch() to create a new record rather than update an existing one.

☑ Exercise 1.3 — Test Submission (Observe the Bug)

💡 **What you are doing:** You are intentionally triggering a known failure so you can see the problem before fixing it. Understanding what failure looks like is just as important as knowing how to fix it.

Enter the following test values:

```
Title    = Printer Failure
Priority = High
```

Click Submit and observe:

- ❌ Record does NOT save
- ❌ No error message appears

🔍 **Why did it fail?** The IncidentCollection was defined with Priority as a NUMBER. You are passing the text "High" — a string — into a numeric field. Power Apps silently drops the record rather than throwing a visible error. This is called a silent failure.

☑ Exercise 1.4 — Fix the Patch Formula

💡 **What you are doing:** You are applying the Value() function to explicitly convert the text input into a number before writing it to the collection. This is called type coercion.

Update the Button OnSelect to:

```
Patch(
    IncidentCollection,
    Defaults(IncidentCollection),
    {
        Title:    txtIncidentTitle.Text,
        Priority: Value(txtPriority.Text)
    }
)
```

💡 **Why Value()?** Value() converts a text string like "3" into the number 3. Without this conversion, the data types are incompatible and Patch silently fails. Always validate and convert input types before writing to a data source.

✅ Record now saves correctly

🔬 Lab 2 — Form Displays "No Item to Display"

🎯 Objective

Correctly bind a form to a gallery selection so the form displays the record the user clicked.

> 💡 **Why this lab exists:** Forms and galleries must be explicitly connected in Canvas Apps — they do not link automatically. This is a top support issue for new developers and a concept that must be understood before building any data-entry app.

☑ Exercise 2.1 — Add a Gallery

> 💡 **What you are doing:** You are adding a gallery control that will display your incident records. The gallery becomes your navigation mechanism — users click a row to select a record.

5. Insert a Vertical Gallery
6. Set the Items property to:

```
IncidentCollection
```

> 💡 **Why set Items?** The Items property tells the gallery where to get its data. Without this, the gallery has nothing to display. Think of it as pointing the gallery to your data source.

☑ Exercise 2.2 — Add a Form (Observe the Bug)

> 💡 **What you are doing:** You are adding an Edit Form control and intentionally leaving the Item binding empty to demonstrate what happens when the form doesn't know which record to show.

7. Insert an Edit Form
8. Set the DataSource property to: IncidentCollection
9. Preview the app
10. Select a gallery item

- ❌ Form shows "No item to display"

> 🔍 **Why did it fail?** Setting DataSource only tells the form what schema to use (what columns exist). It does NOT tell the form which record to display. These two controls are completely unaware of each other until you explicitly connect them.

☑ Exercise 2.3 — Fix the Form Binding

> 💡 **What you are doing:** You are setting the form's Item property to point at the gallery's currently selected record. This is the critical link between navigation and data display.

Set the Form Item property to:

```
Gallery1.Selected
```

💡 **Why Gallery1.Selected?** Every gallery control has a .Selected property that always holds the last record the user clicked. By assigning this to the form's Item, you are saying: "Show whichever record the user selected in the gallery." This is the standard pattern for master-detail forms in Canvas Apps.

✅ Form now displays the selected record's data

🔬 Lab 3 — OnStart Performance Optimization

🎯 Objective

Understand why overloading App.OnStart causes slow startup and learn lazy loading as a performance pattern.

💡 **Why this lab exists:** OnStart is one of the most misused properties in Canvas Apps. Everything placed there runs before the app shows the first screen. In government apps with large datasets or multiple data sources, this can cause 10–30 second load times — a terrible user experience.

☑ Exercise 3.1 — Simulate a Slow Startup

💡 **What you are doing:** You are deliberately adding multiple sequential data loads to App.OnStart to simulate what happens when developers dump all their initialization logic there.

Modify App.OnStart to:

```
ClearCollect(Test1, IncidentCollection);
ClearCollect(Test2, IncidentCollection);
ClearCollect(Test3, IncidentCollection)
```

Reload the app and observe: ⏱ Slow startup — app hangs before showing Screen1.

🔍 **Why is it slow?** Each ClearCollect() call is a sequential operation. The app cannot display anything until every line of OnStart finishes executing. Three collections loading back-to-back creates a compounding delay.

☑ Exercise 3.2 — Move Logic to Screen.OnVisible

💡 **What you are doing:** You are relocating the data loading logic from the app level to the screen level. This means the app can open immediately and load data only when a specific screen becomes visible.

Move the ClearCollect calls from App.OnStart to Screen1.OnVisible:

```
ClearCollect(Test1, IncidentCollection);
ClearCollect(Test2, IncidentCollection);
ClearCollect(Test3, IncidentCollection)
```

💡 **Why OnVisible?** OnVisible fires each time a screen is navigated to. By moving initialization here, the app loads the first screen instantly and only loads data when it's actually needed.

☑ Exercise 3.3 — Apply Lazy Loading

💡 **What you are doing:** You are adding a conditional check so the collection only loads if it's empty. This prevents unnecessary re-loading every time the user returns to the screen.

Update the OnVisible formula to:

```
If(
    IsEmpty(Test1),
    ClearCollect(Test1, IncidentCollection)
)
```

💡 **Why IsEmpty()?** Without this check, ClearCollect() runs every single time the user navigates to the screen — even if the data is already loaded. IsEmpty() acts as a gate: "Only load this data if it hasn't been loaded yet." This is the lazy loading pattern.

✅ Startup time significantly improves.

🔬 Lab 4 — Patch Silent Failure & Error Handling

🎯 Objective

Replace silent failures with user-visible error messages using IfError() and Notify().

💡 **Why this lab exists:** Silent failures are dangerous in production apps. If a record fails to save and the user sees nothing, they may believe the save was successful — leading to data loss. Proper error handling is not optional; it is a production requirement.

☑ Exercise 4.1 — Simulate a Silent Failure

💡 **What you are doing:** You are using SubmitForm() without any error handling to demonstrate what the user experiences when a save operation fails with no feedback.

Set the Save button OnSelect to:

```
SubmitForm(Form1)
```

- ❌ Item does NOT save
- ❌ No error message is shown
- ❌ User has no idea what happened

🔍 **Why is this dangerous?** In a real incident reporting system, a user who thinks their report was submitted — when it actually wasn't — could result in critical issues going unresolved. Silent failures erode trust in the application.

☑ Exercise 4.2 — Add Error Handling with IfError and Notify

> 💡 **What you are doing:** You are wrapping the form submission in IfError() to intercept any failure and surface a message to the user. A success notification is also added so users get confirmation either way.

Update the Save button OnSelect to:

```
IfError(
    SubmitForm(Form1),
    Notify("Error Saving Record — Please Try Again", NotificationType.Error),
    Notify("Record Saved Successfully!",             NotificationType.Success)
)
```

> 💡 **Why IfError()?** IfError() evaluates an expression and branches based on whether it succeeds or fails — similar to try/catch in traditional programming. The first argument is the expression to try. The second argument runs if it fails. The third argument runs if it succeeds.
>
> 💡 **Why Notify()?** Notify() displays a banner message at the top of the screen. Using NotificationType.Error shows it in red; NotificationType.Success shows it in green. These visual cues help users immediately understand what happened.

✅ Error message now displays on failure

✅ Success message confirms when save works

🔬 Lab 5 — Runtime Performance Bottleneck

🎯 Objective

Eliminate repeated runtime recalculations by caching frequently used lookup values in variables.

> 💡 **Why this lab exists:** Canvas Apps recalculate every formula on every screen render. A LookUp() call inside a label recalculates constantly — every time anything on the screen changes. In a busy app, this creates measurable UI lag and a sluggish user experience.

☑ Exercise 5.1 — Simulate a Runtime Bottleneck

> 💡 **What you are doing:** You are placing a LookUp() call directly inside a label's Text property to demonstrate how live formula recalculation creates UI lag.

Insert a Label and set its Text property to:

```
LookUp(
    IncidentCollection,
    ID = 1
).Title
```

🔍 **Why is this slow?** Every time the screen re-renders — which happens whenever any control changes — this LookUp() runs again from scratch. It scans the entire IncidentCollection searching for ID = 1 every single time. In a large collection, this is computationally expensive.

☑ Exercise 5.2 — Cache the Value in a Variable

💡 **What you are doing:** You are loading the lookup result once into a variable, then pointing the label at the variable instead. The lookup only runs when the button is pressed — not on every screen render.

Insert a Button and set its OnSelect to:

```
Set(
    varIncidentTitle,
    LookUp(
        IncidentCollection,
        ID = 1
    ).Title
)
```

Update the Label's Text property to:

```
varIncidentTitle
```

💡 **Why Set() and a variable?** Set() stores a value in a global variable that persists across screens. Reading a variable is near-instantaneous — no collection scan, no formula recalculation. The expensive LookUp() runs once when needed, and the result is reused as many times as required. This is the cache pattern for Canvas Apps performance.

✅ UI responds noticeably faster

✅ Navigation is smooth

🔬 Lab 6 — App Checker

🎯 Objective

Use the built-in App Checker tool to proactively identify formula errors, performance warnings, and accessibility issues before users encounter them.

💡 **Why this lab exists:** App Checker is a free, built-in static analysis tool that scans your entire app for known problems. Professional developers run App Checker before every deployment. Many issues it finds would otherwise only surface at runtime — in front of real users.

☑ Exercise 6.1 — Run App Checker

💡 **What you are doing:** You are using App Checker as a final quality gate to catch anything you may have missed during development.

11. In Power Apps Studio, click the App Checker icon (shield/checkmark icon in the top toolbar)
12. Review the results panel — organized into the categories below:

Category	What to Look For
🔴 Formula Issues	Syntax errors, type mismatches, missing required fields
⚠️ Performance Warnings	Nested LookUps, non-delegable filters, heavy OnStart logic
♿ Accessibility Issues	Missing labels, low color contrast, tab order problems

13. Click each issue to navigate directly to the problematic control
14. Fix all listed problems before proceeding to the Challenge Lab

> 💡 **Pro Tip:** In government and enterprise deployments, accessibility compliance (WCAG 2.1) is often legally required. App Checker's accessibility warnings are not cosmetic — they are compliance issues.

🔬 Lab 7 — Monitor Runtime Behavior

🎯 Objective

Use the Power Apps Monitor tool to capture and analyze live runtime events, data operations, and errors during app execution.

> 💡 **Why this lab exists:** Some bugs only appear at runtime — they cannot be caught by App Checker or formula review alone. Monitor is your live debugger. It records every data operation, network call, and error that occurs while the app runs.

☑ Exercise 7.1 — Open Monitor and Capture Events

> 💡 **What you are doing:** You are launching the app inside the Monitor tool so every operation is recorded and available for inspection.

15. In Power Apps Studio, navigate to: Advanced Tools → Monitor
16. Click Play to launch the app inside the Monitor session
17. Perform the following actions inside the running app:

Action	What to Watch For in Monitor
Submit a Patch operation	Look for the Patch event — check if it succeeded or failed
Submit a Form	Look for SubmitForm event — check response codes
Select a gallery item	Look for data read events and binding updates

In the Monitor panel, observe:

- Runtime errors — red entries indicating failures
- Data operations — reads and writes to collections/data sources
- Network calls — connector calls showing latency and response

Column	Meaning
Category	Type of operation (Behavior, Network, Error)
Control	Which control triggered the event
Property	Which property fired (OnSelect, OnVisible, etc.)
Result	Success or Failure
Duration	How long the operation took in milliseconds

💡 **Pro Tip:** In Monitor, clicking any row expands its full details — including the exact formula that ran, the data payload sent, and the error message returned. This is invaluable for diagnosing Patch failures that produce no visible error in the app itself.

🧪 FINAL CHALLENGE LAB — Independent Troubleshooting Scenario

🎯 **Purpose:** Now that you have learned each concept in isolation, this challenge requires you to apply them together — just like you would in a real-world support or development scenario. No step-by-step guidance is provided. Use everything you have learned, including Monitor and App Checker.

Task	Goal
✓ Fix Patch failure	Identify and resolve a type mismatch
✓ Fix missing gallery data	Diagnose and correct a filter formula
✓ Fix slow startup	Optimize OnStart logic
✓ Prevent silent runtime errors	Add proper error handling
✓ Improve runtime performance	Cache lookup values

☑ Challenge Task 1 — Gallery Missing Records

Modify the Gallery Items property to:

```
Filter(
    IncidentCollection,
    Len(Title) > 5
)
```

- ❌ Some records are missing from the gallery

Your task: Determine why records are being excluded and fix the filter formula so all valid records display.

🔍 **Hint:** Think about what Len() returns and which records in your collection would be affected by this condition.

☑ Challenge Task 2 — Startup Delay

Modify App.OnStart to include:

```
ClearCollect(Test4, IncidentCollection);
ClearCollect(Test5, IncidentCollection)
```

Reload the app and observe the delay. Then apply the optimization patterns from Lab 3 to eliminate the startup bottleneck.

> 🔍 **Hint:** Consider where this logic should live and whether lazy loading applies here.

☑ Challenge Task 3 — Runtime Patch Failure

Modify the Submit button to:

```
Patch(
    IncidentCollection,
    Defaults(IncidentCollection),
    {
        Title:    txtIncidentTitle.Text,
        Priority: txtPriority.Text
    }
)
```

Your task:

18. Open Monitor before testing
19. Attempt a submission
20. Use Monitor to locate the exact failure event
21. Fix the Patch formula

> 🔍 **Hint:** Review what data type Priority expects vs. what txtPriority.Text provides.

☑ Challenge Task 4 — Silent Form Submission

Remove all error handling from the Save button so it reads only:

```
SubmitForm(Form1)
```

Preview the app, attempt to save, and observe the silent failure. Then modify the Save button logic to: display a clear error message when saving fails, confirm success when saving succeeds, and never leave the user without feedback.

> 🔍 **Hint:** Refer to the pattern you implemented in Lab 4.

☑ Challenge Task 5 — Runtime Optimization

Add a second label to the screen with a LookUp() formula targeting a different record ID. Then optimize the formula to prevent UI lag using the caching pattern from Lab 5.

> 🔍 **Hint:** Consider when and where the variable should be set for maximum efficiency.

🏁 Workshop Completion Checklist

You have successfully completed this workshop when all items below are checked:

Checkpoint	Status
✅ Patch operations save records correctly	☐
✅ Form displays the selected gallery record	☐
✅ App startup delay is eliminated or reduced	☐
✅ Gallery loads all expected records	☐
✅ Runtime errors are visible to the user	☐
✅ UI performance is smooth and responsive	☐
✅ Monitor shows no unhandled failures	☐
✅ App Checker shows no critical issues	☐

🎓 **Congratulations!** Congratulations! You have completed the full end-to-end Canvas Apps Troubleshooting Workshop. The skills practiced here — type-safe Patch operations, proper form binding, performance optimization, error handling, and runtime diagnostics — are the foundation of production-quality Power Platform development in any enterprise or government environment.